

Deadline. The homework is due **11:59pm, Mon 05/11, 2026**. You must submit your solutions (in pdf format generated by LaTeX) via GradeScope.

Late Policy. You have up to four grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can get help from the instructor, TAs, or your classmates, but only after you have thought about the problems on your own. **If you discussed with anyone, you need to acknowledge them in your report.**
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. **If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.**
- **AI Tools.** If you use ChatGPT or similar AI tools, you have to attach the full conversation to make it clear what help you obtained from them.

However, you **CANNOT**:

- share your report/code with anyone else,
- read other's report/code,
- copy anything from other source,
- get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article/AI tools such as ChatGPT) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason. **In summary, you must write up your solution independently, and type your answers word by word on your own: close the book/notes/online resources when you are writing your final answers. Add all the citations and collaborators' names to the end.**

Write-up. Please use LaTeX to prepare your solutions. For all problems, **please explain how you get the answer** instead of directly giving the final answer, except for some special cases (will be specified in the problem). For all algorithm design problems, please describe using **natural language**. You can present pseudocode if you think that helps to illustrate your idea, but please do not only give some code without explanation. **In grading, we will reward not only correctness but also clarity and simplicity. To avoid losing points for hard-to-understand solutions, you should make sure your solutions are either self-explanatory or contains good explanations.** See some more details here <https://www.cs.ucr.edu/~yihans/teaching/214/S26/assignments/>

Bonus questions. Bonus questions are not required, and the points are not counted in the basic 100 points of the class. Most of the bonus questions are just slightly harder than the regular questions, but some can be extremely hard. In a nutshell, most of them are worthy trying, but don't be frustrated if you cannot solve *all* of them :) We will not provide too many hints/solutions for bonus problems, but feel free to discuss them with your classmates.

1 (2 pts) Fibonacci Numbers

In this question, we will try to generate the first n Fibonacci numbers in parallel. Recall that $F_0 = 0, F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$ for $i \geq 2$.

Questions:

- (0.5 pt) First, define a vector $v_i = \begin{bmatrix} F_{i-1} \\ F_i \end{bmatrix}$ to denote the $(i-1)$ -th and i -th Fibonacci number. How can we represent v_{i+1} using v_i using a transition matrix? In other words, find a 2×2 matrix A , such that $A \cdot v_i = v_{i+1}$ for all i .
- (1.5 pts) Give a parallel algorithm to compute $F_1, F_2, \dots, F_n$. Your algorithm must take $O(n)$ work and $O(\log n)$ span. Please prove the cost of your algorithm. (Hint: you can first find a formula that computes v_i by A and v_1 for any i .)

2 (4 pts + 1.5 bonus pts + 1.5 candies) Parallel Merging

In class we've seen a parallel merge sort algorithm using $O(\log^3 n)$ span. This is because the merging algorithm itself takes $O(\log^2 n)$ span. In this problem, we'll design some other merging algorithms with lower span.

For simplicity, in the following problems you can assume all keys are distinct.

(0.5 bonus pt + 0.5 candy) The subroutine: select the k -th element

Assume you are given two arrays A and B , each with keys in sorted order, and an integer $k, 0 \leq k < |A| + |B|$. Let x_k denote the k -th smallest element out of the $n + m$ total elements. Describe an algorithm that returns the cut point in each array such that all elements below the cut point are less than or equal to x_k , and all elements above are greater than x_k . Your algorithm can be sequential but must only take $O(\log(|A| + |B|))$ work (sequential time).

You will use this algorithm as a black box in the following questions. **If you don't choose to work on this bonus question, you can just assume that we can use such a black box that finds the k -th element in $O(\log(|A| + |B|))$ work.**

Questions:

- (bonus 0.5 pt + 0.5 candy) Describe an algorithm that, given two sorted arrays A and B , return the k -th element in $A \cup B$ in $O(\log(|A| + |B|))$ work. Prove the cost of your algorithm.

(4 pts + 1 bonus pt + 1 candy) Parallel Merging in $O(\log n)$ Span

To get rid of an extra factor of $O(\log n)$ in the span bound, the idea is to use multi-way divide-and-conquer instead of two-way (see an example in Figure 1). Next, you will design two different algorithms for parallel merging.

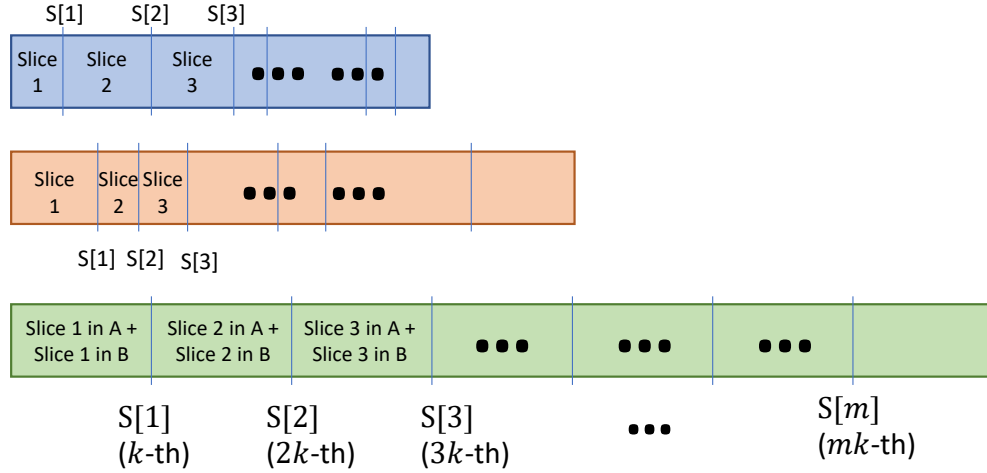


Figure 1: The multi-way merging algorithm.

(2.5 pts) Idea 1: Blocking + Sequential Merge

The first algorithm works as follows.

- Step 1. For two arrays A and B with sizes $|A|$ and $|B|$, let $k > 0$ be a parameter and $m = \lfloor (|A| + |B|)/k \rfloor$. Assume the output will be stored in array C .
- Step 2. First we need to find the global k -th, $2k$ -th, ... $m \cdot k$ -th elements in the two sorted array, using the select k -th element algorithm above. Store these elements as $s[1..m]$ and call them *pivots*.
- Step 3. The pivots split each of the two arrays into $m + 1$ slices.
- Step 4. Start m tasks in parallel, the i -th task is to merge the i -th slice in A and the i -th slice in B , **sequentially**. The i -th task will write the results in $C[ik..(i+1)k - 1]$.

We can easily see the correctness of this algorithm. Next let's prove its cost bounds.

Questions:

2. (1 pt) Write down the work and the span of this algorithm, respectively, using parameter k .
3. (1.5 pt) Choose the value of parameter k to make the algorithm work-efficient (i.e., $O(n)$ work) and bound the span in $O(\log(|A| + |B|))$. Show how you find the value of k and prove the cost of the algorithm.

(1.5 pts + 1 bonus point + 1 candy) Idea 2: Blocking + Recursive Merge

The second algorithm works as follows.

- Step 1. For two arrays A and B with sizes $|A|$ and $|B|$, let $k > 0$ be a parameter and $m = \lfloor (n_a + n_b)/k \rfloor$. Assume the output will be stored in array C .

- Step 2. First we need to find the global k -th, $2k$ -th, ... $m \cdot k$ -th elements in the two sorted array, using the select k -th element algorithm above. Note these elements as $s[1..m]$ and call them pivots.
- Step 3. The pivots split each of the two arrays into $m + 1$ slices.
- Step 4. Start m tasks in parallel, the i -th task is to merge the i -th slice in A and the i -th slice in B , **recursively**. Write the results in $C[ik..(i + 1)k - 1]$.

Questions:

4. (1 pt) Write down the recurrence to compute the work and the span of this algorithm, respectively.
5. (0.5 pts) Prove that if we choose $k = \Theta(\sqrt{n})$, the span is $O(\log(|A| + |B|))$.
6. (Bonus 1 pt + 1 candy) Prove that if we choose $k = \Theta(\sqrt{n})$, the work is $O(n)$.

3 (1 pt) Friend Recommendation

Many social networks provide services of friend recommendation. While there are many factors to consider, a straightforward idea is to use the structure information of the network. Usually, we can recommend the users' friend's friend to them. In this task, you are going to implement such a recommendation system.

In particular, you can access the social network by using the following APIs:

Function	Description
get_friends(user u)	return an array A that contains all friend ids of user u
num_friends(user u)	return the number of friends of user u

Assume each of the functions above only needs constant time to return. All users are denoted uniquely by an integer id in range $[0, n)$, where n is the number of users in this social network. There are m edges in this social network, and all friendship relationships are bi-directional.

Using these APIs, design a parallel algorithm that, given a user u , return an array R and its size, which is all u 's friends' friends (2-hop friends). Note that if a vertex v is u 's direct friend then v shouldn't appear in the output since we don't need to recommend v to u . Obviously, u itself shouldn't appear in the output.

Your algorithm needs to be in time $O(m)$ work and $O(\log n)$ span.

1. (1 pt) Describe your algorithm for the above friend-recommendation task. Your algorithm must have $O(m)$ work and $O(\log n)$ span.

4 (1 pt) With High Probability and Hash Table

In class, we talked about the concept of "with high probability". In particular, we say $f(n) = O(g(n))$ with high probability in n if there exists a constant c , s.t., $\Pr[f(n) \geq k \cdot c \cdot g(n)] \leq \frac{1}{n^k}$, for any $k > 1$.

Then we also learned parallel hash table. To insert some element e into a hash table, we will select a random slot in the hash table, and try to throw it there. If the slot is already taken, called a **collision**, we just repeat the process and select *another* random slot, until an empty slot is found, and we put e into the random slot. We call such a process a **retry**. Note that here we simplified the problem, and assume we always select a new random slot (independently with the previous choice), instead of linear probing. We also assume the load factor (the ratio of taken slots in the hash table) is smaller than $1/2$.

Questions:

For an insertion, assume t is the number of retries. It's easy to see that t is a random variable. Prove that, $t = O(\log n)$ with high probability.